

ЛАБОРАТОРНАЯ РАБОТА № 12

Авторизация и регистрация

Цель: изучить технологию авторизации и регистрации пользователей.

Содержание отчета: титульный лист, цель работы, задание, описание хода выполнения задания со скриншотами, листинги, вывод.

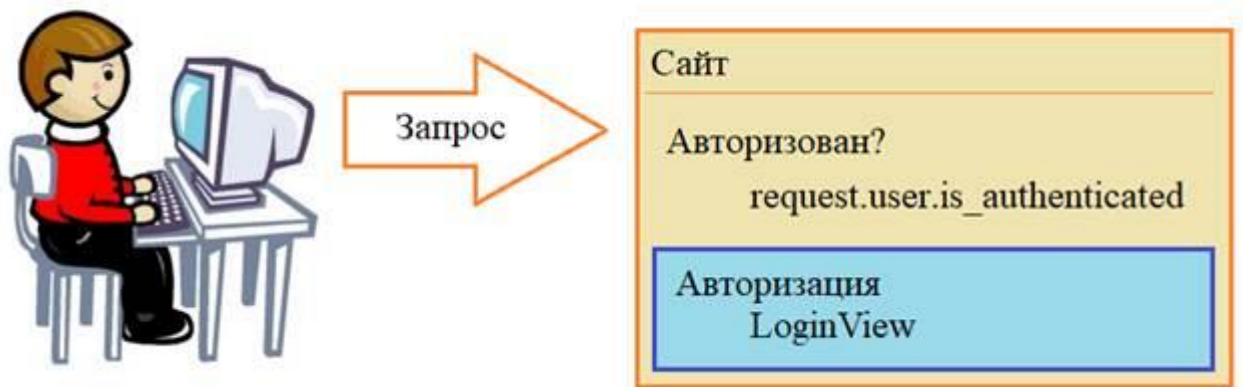
Введение в авторизацию пользователей

Важный раздел фреймворка Django – авторизация пользователей на сайте. Благодаря авторизации пользователя сервер имеет возможность выдавать ему закрытую (персональную) информацию, предназначенную только для этого пользователя и никакого другого. В этом главный смысл и цель создания механизма авторизации.

Немного терминологии:

- **Authentication (аутентификация)** - проверка подлинности (наличия) пользователя по введенным данным (обычно, это логин/пароль). Как правило, это ассоциация (связь) введенных данных с пользователем, сохраненным в БД.
- **Authorization (авторизация)** – разрешение на доступ (непосредственно вход) к закрытой части сайта. Часто на этом этапе выполняется сохранение в сессии идентификатора пользователя, говорящий о том, что пользователь успешно прошел аутентификацию.

Как используется механизм авторизации пользователя? Когда на сервер приходит запрос от пользователя, то на стороне сайта (сервера) важно знать авторизован уже этот пользователь или нет. То есть он мог ранее войти на сайт, набрав логин и пароль. Фреймворк Django хранит эту информацию в сессии и при поступлении очередного запроса от этого же пользователя он воспринимается как авторизованный и автоматически аутентифицируется.



Если же пользователь не авторизован, то на сайте должна отобразиться форма авторизации, где пользователю предлагается ввести логин и пароль:

The image shows a web form for user authentication. It has a title 'Авторизация' (Authorization) at the top. Below the title are two input fields: the first is labeled 'имя пользователя' (username) and the second is labeled 'пароль' (password). Below these fields is a blue button with the text 'Войти' (Login).

Подробную информацию об авторизации, можно посмотреть на странице документации по следующей ссылке:

<https://docs.djangoproject.com/en/4.2/topics/auth/default/>

Таблица user

При установке и создании проекта фреймворк Django сразу формирует набор таблиц для хранения информации по пользователям. Если открыть программу SQLiteStudio, то мы увидим таблицы с префиксом **auth** и среди них таблицу **auth_user**. Как раз здесь хранятся записи по пользователям нашего сайта. Откроем эту таблицу и увидим ее структуру с полями **password**, **username**, **email**, **is_active** и другими. И имеется одна запись для суперпользователя, которого мы ранее создавали. В процессе аутентификации по полям **username** и **password** фреймворк Django выполняет связывание конкретной записи из этой таблицы. Если запись с указанными данными находится, то пользователь считается аутентифицированным. Иначе у него не будет доступа к закрытым страницам сайта.

Помимо таблицы **auth_user** есть и другие вспомогательные таблицы, в частности, **auth_permission** для разрешений зарегистрированных пользователей; таблица **auth_group** для хранения информации о группах пользователей. И некоторые другие. Все это часть стандартного модуля авторизации фреймворка Django. Конечно, его можно настраивать под свои нужды и расширять. В частности, нередко можно встретить расширение стандартной таблицы для хранения зарегистрированных пользователей, так как существующих полей таблицы **auth_user** бывает просто недостаточно.

Создание приложения авторизации

Модуль авторизации логично рассматривать как расширение функционала уже существующего сайта. Поэтому мы его реализуем отдельным приложением с именем **users**. Для этого в терминале выполним команду:

```
python manage.py startapp users
```

Затем, в пакете конфигурации в файле **settings.py** подключим это приложение, прописав его в коллекции **INSTALLED_APPS**:

```
INSTALLED_APPS = [
    ...
    "users.apps.UsersConfig",
]
```

Также обратите внимание, что в списке **MIDDLEWARE** должны присутствовать модули:

```
MIDDLEWARE = [
    ...
    'django.contrib.sessions.middleware.SessionMiddleware',
    ...
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    ...
]
```

Они автоматически срабатывают при каждом запросе-ответе и необходимы для работы модуля авторизации фреймворка Django, основанного на сессиях.

Следующим шагом мы создадим файл **urls.py** в рамках текущего приложения для определения списка маршрутов. Начальное содержимое этого файла будет следующим (по аналогии с файлом из приложения **women**):

```
from django.urls import path
from . import views

urlpatterns = [
    path('login/', views.login_user, name='login'),
    path('logout/', views.logout_user, name='logout'),
]
```

Здесь у нас два маршрута. Первый для отображения и обработки формы авторизации пользователя, а второй – для выхода пользователя.

Соответственно, в файле **views** текущего приложения необходимо прописать функции представления **login_user()** и **logout_user()**. Сделаем это пока следующим образом:

```
def login_user(request):
```

```
return HttpResponseRedirect("login")

def logout_user(request):
    return HttpResponseRedirect("logout")
```

И последним шагом нам нужно связать маршруты в файле **users/urls.py** с фреймворком Django, то есть прописать их в файле **urls.py** пакета конфигурации. Сделаем это следующим образом:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('women.urls')),
    path('users/', include('users.urls',
namespace="users")),
    path("__debug__/", include("debug_toolbar.urls")),
]
```

Обратите внимание, в функции **include()** прописан параметр **namespace**, который позволяет формировать пространство имен для доступа к URL-маршрутам. Например, сейчас для доступа к маршруту

<http://127.0.0.1:8000/users/login/>

достаточно будет обратиться по его имени с указанием пространства имен **users** следующим образом:

[users:login](#)

Таким образом, мы дополнительно изолируем приложение **users** от возможных конфликтов в именах маршрутов других приложений.

Однако если сейчас попытаться запустить тестовый веб-сервер, то получим ошибку, что не определена специальная переменная **app_name**. Она должна присутствовать в файле **users/urls.py**:

```
app_name = "users"
```

После этого веб-сервер должен запуститься и мы видим, что все наши новые маршруты работают.

Авторизация пользователей

Для начала пропишем логику полностью вручную. Начнем с создания формы отображения ввода логина и пароля. Для этого создадим файл **forms.py** в приложении **users** и объявим в нем класс **LoginUserForm** следующим образом:

```

from django import forms

class LoginUserForm(forms.Form):
    username = forms.CharField(label='Логин',
    widget=forms.TextInput(attrs={'class': 'form-input'}))
    password = forms.CharField(label='Пароль',
    widget=forms.PasswordInput(attrs={'class': 'form-
input'}))

```

Мы описали форму, несвязанную с моделью с двумя полями для ввода логина и пароля.

Следующим шагом сформируем шаблон для отображения этой формы. Так же, как и в приложении **women**, создадим подкаталог **templates**, а в нем подкаталог **users**. И уже здесь поместим файл **login.html** со следующим содержимым:

```

{% extends 'base.html' %}

{% block content %}
<h1>Авторизация</h1>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <p><button type="submit">Войти</button></p>
</form>

{% endblock %}

```

Затем в файле **users/views.py** функцию **login_user()** перепишем в следующем виде:

```

def login_user(request):
    form = LoginUserForm()
    return render(request, 'users/login.html', {'form':
form})

```

После запуска веб-сервера на странице:

<http://127.0.0.1:8000/users/login/>

увидим отображение формы для входа. Однако нет отображения главного меню. Но мы это пока оставим и сосредоточимся на функции авторизации.

Реализация авторизации пользователя

Так как форма отправляет данные по POST-запросу, то в функции **login_user()** сначала сделаем проверку на этот тип запроса, а затем следующую логику обработки принятых данных:

```
def login_user(request):
    if request.method == 'POST':
        form = LoginUserForm(request.POST)
        if form.is_valid():
            cd = form.cleaned_data
            user = authenticate(request,
username=cd['username'], password=cd['password'])
            if user and user.is_active:
                login(request, user)
                return
HttpResponseRedirect(reverse('home'))
        else:
            form = LoginUserForm()
            return render(request, 'users/login.html', {'form':
form})
```

Мы здесь сначала создаем форму **LoginUserForm** с набором принятых данных **request.POST**. Затем проверяем форму на корректность (валидность) и, если проверка проходит, то создаем временную переменную **cd**, которая ссылается на очищенные принятые данные формы. На основе полей **username** и **password** мы пытаемся с помощью функции **authenticate()** аутентифицировать пользователя по таблице **user** БД. Если пользователь с указанной парой логин/пароль находится в БД и является активным (не забанен, например), то вызывается ключевая функция **login()**, которая создает запись в сессии, авторизуя текущего пользователя на сайте. После этого делается перенаправление на главную страницу. В случае каких-либо ошибок снова форма отображается в браузере пользователя, предлагая ему еще раз попробовать ввести логин и пароль.

Посмотрим, как это все будет работать. На данный момент у нас имеется только один пользователь (он же суперпользователь). Вводим неверные данные и видим, что форма отобразилась снова, так как пользователь не был найден в БД. Если же ввести верные логин/пароль, то произойдет перенаправление на главную страницу.

Видим, что функционал в базовом варианте работает. Но нам пока не показываются никакие сообщения о возможных ошибках. На данном этапе оставим все как есть и реализуем функционал по выходу пользователя (logout).

Перейдем в функцию **logout_user()** и вызовем в ней стандартную функцию **logout()** фреймворка Django с перенаправлением на форму авторизации:

```
def logout_user(request):  
    logout(request)  
    return HttpResponseRedirect(reverse('login'))
```

Давайте ее протестируем. Перейдем по адресу:

127.0.0.1:8000/users/logout/

и мы оказались не на той странице, так как имя **login** связано с маршрутом приложения **women**. Нам здесь нужно дополнительно прописать пространство имен **users** следующим образом:

```
return HttpResponseRedirect(reverse('users:login'))
```

Теперь все работает как задумывалось.

Проверим, действительно ли пользователь вышел из системы. Откроем страницу админ-панели:

<http://127.0.0.1:8000/admin/>

и мы автоматически были перенаправлены на страницу авторизации. Значит, выход действительно был выполнен. Функция представления **logout_user()** работает.

Шаблонные контекстные процессоры

Мы сделали первый шаг по авторизации пользователей на сайте. Однако в шаблоне авторизации у нас не отображается главное меню. Давайте поправим этот момент, прежде чем двигаться дальше.

Мы помним, что приложения в проекте Django следует делать максимально независимыми друг от друга. Поэтому отображение меню лучше сделать непосредственно в базовом шаблоне, передавая ему объект-меню, например, в виде списка. Этот функционал можно реализовать несколькими способами. Используя только текущие знания, первое, что можно сделать, это воспользоваться простым пользовательским тегом. Перейдем в файл **women_tags.py** и определим simple тег для проброса списка меню в шаблон:

```
@register.simple_tag  
def get_menu():  
    return menu
```

А в шаблоне **base.html** воспользуемся им:

```
{% get_menu as menu %}
```

Теперь отдельно меню передавать в шаблон не нужно, поэтому в файле **women/utils.py** уберем его из класса **DataMixin**. Обновляем страницу, видим отображение главного меню.

Однако, когда предполагается во все шаблоны проекта передавать какую-либо независимую информацию, как, например, главное меню, то можно воспользоваться еще одним механизмом — шаблонными контекстными процессорами. Что это такое? Если открыть файл **settings.py**, то в коллекции **TEMPLATES** в списке **context_processors** видим перечисление существующих контекстов. Например, **context_processors.request** формирует переменную **request** во всех наших шаблонах, а **auth.context_processors.auth** обеспечивает переменную **user**. И нам ничто не мешает создать свой собственный шаблонный контекстный процессор для передачи списка пунктов главного меню.

В приложении **users** создадим еще один вспомогательный файл с именем **context_processors.py** и в нем создадим свой контекст для шаблона следующим образом:

```
from women.utils import menu

def get_women_context(request):
    return {'mainmenu': menu}
```

Осталось его прописать в списке других контекстных процессоров (в файле **settings.py**):

```
'context_processors': [
    'django.template.context_processors.debug',
    'django.template.context_processors.request',
    'django.contrib.auth.context_processors.auth',
    'django.contrib.messages.context_processors.messages',
    'users.context_processors.get_women_context'
],
```


Теперь во все наши шаблоны будет автоматически передаваться переменная **mainmenu** со списком главного меню. Используем ее в шаблоне **base.html** вместо переменной **menu**. А тег, который пробрасывал переменную **menu** в шаблон можно убрать.

Так можно создавать свои контекстные процессоры для автоматической передачи глобальной информации в наши шаблоны. Однако, использовать этот механизм следует только для тех данных, которые передаются во все (или почти все) шаблоны. Отправлять таким образом какие-либо частные данные не нужно.

Доработка панели главного меню

Сделаем еще одно изменение. Удалим из списка **menu** последнюю строку для **login**. Мы пропишем ее отдельно в шаблоне. Откроем файл **base.html** и меню будем формировать следующим образом:

```
{% block mainmenu %}
    <div class="header">
        <ul id="mainmenu"
class="mainmenu">
            <li class="logo"><a
href="{% url 'home' %}"><div
class="logo"></div></a></li>
{% for m in mainmenu %}
            <li><a href="{% url
m.url_name %}">{{m.title}}</a></li>
{% endfor %}
            <li class="last"><a
href="{% url 'users:login' %}">Войти</a></li>
        </ul>
        <div class="clear"></div>
    </div>
{% endblock mainmenu %}
```

Обратите внимание, как указано имя маршрута для **login**. Сначала идет пространство имен **users**, а через двоеточие – имя **login**. Это пространство имен мы определили в файле **urls.py** пакета конфигурации, когда подключаем через функцию **include()** файл **users/urls.py**. Благодаря этому, мы на уровне шаблонов можем четко разграничивать имена маршрутов для приложения **users** и других программных компонент. В частности, если прописать просто имя **login**, то получим другой маршрут с этим именем, прописанный в файле **women/urls.py**.

Теперь отобразим в главном меню имя пользователя, если он авторизован, либо пункт «Войти» и «Регистрация», если не авторизован. Для этого в главном шаблоне **base.html** пропишем следующую проверку:

```
{% if user.is_authenticated %}
<li class="last"> {{user.username}} | <a href="{% url
'users:logout' %}">Выйти</a></li>
{% else %}
<li class="last"><a href="{% url 'users:login'
%}">Войти</a> | <a href="#">Регистрация</a></li>
{% endif %}
```

Здесь используется объект **user**, связанный с текущим пользователем, и уже у этого объекта проверяем свойство **is_authenticated**. Если оно принимает значение **True**, значит, пользователь авторизован, иначе – не авторизован. Для авторизованных пользователей отображается ссылка «Выйти» с именем маршрута **logout**. А для неавторизованных ссылки «Войти» и «Регистрация».

Нажмем на ссылку «Войти» и в форме авторизации введем данные суперпользователя. На панели отобразилось имя пользователя и пункт «Выйти». Нажимаем на выход, попадаем снова на страницу авторизации.

Классы **LoginView** и **AuthenticationForm**

Мы вручную реализовали механизм авторизации пользователей на сайте. Теперь перепишем его с использованием стандартных классов, поставляемых фреймворком Django. Они следующие:

- **LoginView** – класс представления для авторизации пользователей;
- **LogoutView** – класс представления для выхода пользователя из системы;
- **AuthenticationForm** – класс формы обработки аутентификации пользователя.

Начнем с класса **LoginView**, которым заменим функцию **login_user()** следующим образом:

```
class LoginUser(LoginView):
    form_class = AuthenticationForm
    template_name = 'users/login.html'
    extra_context = {'title': "Авторизация"}
```

Нам уже знакомы все используемые атрибуты. Обратите внимание, что атрибуту **form_class** мы присваиваем стандартный класс формы **AuthenticationForm** фреймворка Django. Если здесь прописать наш текущий класс формы:

```
form_class = LoginUserForm
```

то получим ошибку, так как наше новое представление **LoginUser** ожидает форму с набором определенных методов и атрибутов, чтобы в связке с ней обрабатывать процесс авторизации. Конечно, мы можем вручную создать нужную форму, но проще воспользоваться уже имеющейся для этих целей:

```
form_class = AuthenticationForm
```

Либо, при необходимости, расширить ее дочерним классом. Пока оставим наше представление в таком виде. Подключим его к маршруту **login** в файле **users/urls.py**:

```
urlpatterns = [  
    path('login/', views.LoginUser.as_view(),  
name='login'),  
    path('logout/', views.logout_user, name='logout'),  
]
```

Запустим веб-сервер и перейдем в форму авторизации. Все отображается и при вводе логина и пароля происходит автоматическое перенаправление в профайл по URL-адресу:

<http://127.0.0.1:8000/accounts/profile/>

Но нам это не нужно. Изменим адрес перенаправления, переопределив метод **get_success_url()** в классе **LoginUser**:

```
def get_success_url(self):  
    return reverse_lazy('home')
```

Теперь, при авторизации будем попадать на главную страницу. Тот же самый эффект можно получить, определив константу:

```
LOGIN_REDIRECT_URL = '/'
```

в файле **settings.py** пакета конфигурации **sitewomen**. Но лучше здесь указывать имена маршрутов:

```
LOGIN_REDIRECT_URL = 'home'
```

Вообще, в файле конфигурации **settings.py** можно прописывать следующие параметры:

- **LOGIN_REDIRECT_URL** – задает URL-адрес, на который следует перенаправлять пользователя после успешной авторизации;

- **LOGIN_URL** – определяет URL-адрес, на который следует перенаправить неавторизованного пользователя при попытке посетить закрытую страницу сайта;
- **LOGOUT_REDIRECT_URL** – задает URL-адрес, на который перенаправляется пользователь после выхода.

Улучшение формы авторизации

Класс представления **LoginView** берет на себя значимую часть стандартной работы по авторизации пользователя. И дополнительно он формирует сообщения об ошибках, которые передает в форму, при неверно заполненных данных. Давайте в этом убедимся. Для этого откроем шаблон формы **login.html** и вывод полей формы будем делать через цикл:

```
<div class="form-error">{{ form.non_field_errors }}</div>

{% for f in form %}
<p ><label class="form-label" for="{{ f.id_for_label }}">{{f.label}}: </label>{{ f }}</p>
<div class="form-error">{{ f.errors }}</div>
{% endfor %}
```

Не забывайте сначала делать отображение общих ошибок коллекции **form.non_field_errors**.

Обновим страницу авторизации и введем неверную пару логин/пароль. Увидим отображение ошибки. Эта ошибка была автоматически сформирована классом представления **LoginView**. И это очень удобно. Обработку всех стандартных ситуаций фреймворк Django берет на себя.

Но мы бы хотели еще улучшить и сам класс формы. Для этого в файле **users/forms.py** существующий класс **LoginUserForm** унаследуем от класса **AuthenticationForm** следующим образом:

```
class LoginUserForm(AuthenticationForm):
    username = forms.CharField(label='Логин',
    widget=forms.TextInput(attrs={'class': 'form-input'}))
    password = forms.CharField(label='Пароль',
    widget=forms.PasswordInput(attrs={'class': 'form-
input'}))
```

Обновим форму авторизации и видим, что в целом все работает. Обратите внимание, что в этой форме поля должны называться именно так, как прописаны: **username** и **password**. Вообще, они должны соответствовать стандартной модели пользователя **User** фреймворка Django. Поэтому, как

вариант, мы могли бы определить этот же класс **LoginUserForm** следующим образом:

```
class LoginUserForm(AuthenticationForm):
    class Meta:
        model = get_user_model()
        fields = ['username', 'password']
```

Мы здесь получаем модель **User** с помощью стандартной функции **get_user_model()**. Это рекомендуемая практика на случай изменения модели. Тогда в программе ничего дополнительно менять не придется. Также указали отображать в форме поля **username** и **password**. Именно они необходимы для аутентификации пользователя по БД.

В общем случае, можно комбинировать мета-описание и отдельные атрибуты:

```
class LoginUserForm(AuthenticationForm):
    username = forms.CharField(label='Логин',
        widget=forms.TextInput(attrs={'class': 'form-input'}))
    password = forms.CharField(label='Пароль',
        widget=forms.PasswordInput(attrs={'class': 'form-input'}))

    class Meta:
        model = User
        fields = ['username', 'password']
```

Так форма выглядит понятнее по своей логике работы.

Параметр next

Последний штрих, который мы сделаем в форме – это добавим скрытое поле с именем **next** в шаблон **login.html** следующим образом:

```
<input type="hidden" name="next" value="{{ next }}" />
```

Что это за параметр? Дело в том, что теперь мы можем в URL-адресе формы авторизации дополнительно прописывать этот параметр **next**, например, так:

<http://127.0.0.1:8000/users/login/?next=/addpage/>

Он содержит URL-адрес страницы, на которую следует сделать перенаправление после успешной авторизации. В данном случае, мы попадаем на страницу добавления статей. Причем этот параметр **next** имеет более высокий приоритет, чем параметр **LOGIN_REDIRECT_URL**, прописанный в пакете конфигурации.

Зачем может понадобиться этот параметр? Когда вы выбираете товар в интернет-магазине, для его помещения в корзину необходима авторизация. Форма ввода логина/пароля сразу появляется на экране и после корректного ввода, вы сразу попадаете в корзину. Как сайт догадался вас перенаправить именно в корзину? Сработал аналог параметра **next** для перенаправления на нужную страницу сайта. И то же самое могло произойти, когда товар уже в корзине, а для оформления заказа понадобилась авторизация. Тогда посетитель перенаправляется уже не в корзину, а на оформление заказа. То есть иногда нам нужна страница после авторизации, адрес которой можно сохранить в параметре **next**.

Класс LogoutView

С помощью класса LogoutView можно заменить функцию представления **logout_user()**. Если нам достаточно стандартного поведения, то класс **LogoutView** можно сразу связать с нужным маршрутом (в файле **users/urls.py**):

```
urlpatterns = [
    path('login/', views.LoginUser.as_view(),
name='login'),
    path('logout/', LogoutView.as_view(),
name='logout'),
]
```

А функцию **logout_user()** просто убрать. Переходим на сайт, нажимаем «Выйти» и попадаем на несуществующую страницу:

<http://127.0.0.1:8000/users/logout/>

Как нам задать другой адрес для выхода? Для этого можно воспользоваться константой **LOGOUT_REDIRECT_URL**, прописав ее в файле **settings.py**, например, так:

```
LOGOUT_REDIRECT_URL = 'home'
```

Теперь мы будем выходить на главную страницу сайта.

Декоратор login_required

На данный момент мы научились авторизовывать существующих пользователей на сайте. Но все страницы сайта как были открытыми для посетителей, так и остаются в открытом виде. Давайте добавим ограничения на некоторые из них.

Начнем с функций представлений. В файле **women/views.py** у нас имеется такая подходящая функция с именем **about()**, которая отвечает за страницу «О сайте». Чтобы закрыть ее от неавторизованных пользователей, достаточно декорировать ее декоратором **login_required** следующим образом:

```
@login_required
def about(request):
    ...
```

Если теперь запустить веб-сервер и попытаться перейти на эту страницу неавторизованному пользователю, то выполнится автоматическое перенаправление по адресу:

<http://127.0.0.1:8000/accounts/login/?next=/about/>

Обратите внимание на появление параметра **next**. Благодаря ему, после успешной авторизации мы сразу перейдем к закрытой странице «О сайте». Но на данный момент у нас другая проблема. Фреймворк Django выполняет автоматическое перенаправление по несуществующему URL-адресу. Поправим это. Вспомним об еще одном параметре **LOGIN_URL**, который определяет адрес страницы с формой авторизации. В файле **settings.py** пропишем его следующим образом:

```
LOGIN_URL = 'users:login'
```

Теперь при заходе на страницу «О сайте» нам открывается форма авторизации. И фреймворк Django автоматически добавляет к URL-адресу параметр **next**. Вводим верные логин/пароль, и эта страница отображается в браузере. Вот так с помощью декоратора **login_required** можно ограничивать доступ к отдельным страницам сайта.

Другой вариант перенаправления при доступе к закрытой странице можно определять с помощью параметра **login_url** самого декоратора:

```
@login_required(login_url='/admin/')
...
```

Произойдет перенаправление к странице админ-панели. Причем, приоритет этого параметра выше, чем **LOGIN_URL**.

Класс миксин **LoginRequiredMixin**

Когда мы имеем дело с классами представлений, то вместо декоратора **login_required** используется класс минксинов **LoginRequiredMixin**. В данном случае **LoginRequiredMixin** добавляет новый функционал к классу

представления, ограничивая доступ к странице неавторизованным пользователям. Давайте его пропишем для класса **AddPage**, предполагая, что новые статьи могут добавлять только зарегистрированные пользователи:

```
class AddPage(LoginRequiredMixin, DataMixin,
CreateView):
    ...
```

Теперь при попытке открыть эту страницу сторонним посетителем, будет выполнено перенаправление на форму авторизации. URL-адрес этой формы также будет содержать параметр **next**. После ввода логина/пароля попадаем на страницу добавления статьи.

Если нам нужно в классе представления определить свой URL-адрес перехода для неавторизованных пользователей, то достаточно прописать атрибут **login_url**, например, так:

```
login_url = '/admin/'
```

При обращении к закрытой странице выполнится перенаправление на админ-панель. И так можно ограничивать доступ к любым страницам сайта. Но сейчас уберем этот последний параметр.

Добавление авторства статей

Поскольку мы сделали ограничение доступа к странице добавления статей, то было бы логично с каждой статьей связать своего автора. Для этого в модели **Women** мы добавим еще одно поле в виде связи многие к одному следующим образом:

```
author = models.ForeignKey(get_user_model(),
on_delete=models.SET_NULL, related_name='posts',
null=True, default=None)
```

Мы обращаемся к модели **User** с помощью функции **get_user_model()**. Это считается предпочтительной практикой для фреймворка Django. Затем прописываем параметр **on_delete** с флагом **SET_NULL**, то есть, при удалении пользователя поле **author** будет содержать значение **NULL**. Остальные параметры нам уже известны.

Так как модель изменилась, то нужно создать и применить миграции:

```
python manage.py makemigrations
python manage.py migrate
```

Таблица **women** обновлена, и в ней появилось новое поле **author_id** со значениями **NULL**. Осталось только заполнять это поле в момент добавления

статьи. Для этого перейдем в файл **women/views.py** и в класс **AddPage** добавим метод **form_valid()**:

```
def form_valid(self, form):
    w = form.save(commit=False)
    w.author = self.request.user
    return super().form_valid(form)
```

Мы здесь обращаемся к форме и вызываем ее метод **save()** с параметром **commit=False**. Благодаря ему, данные не записываются в таблицу БД, а формируются только в памяти. В результате ссылка **w** будет указывать на объект модели **Women** текущей записи. Меняем в этом объекте атрибут **author**, присваивая ему объект **user** текущего пользователя. Измененный объект **w** будет автоматически сохранен в таблице БД.

Посмотрим, как это будет работать. Откроем страницу добавления записи. Введем произвольные данные и видим, что в поле **author_id** таблицы **women** появилось значение **1**. Это как раз идентификатор авторизованного суперпользователя.

Поскольку каждая статья связана с пользователем, то отобразим его **username** в списке статей. Откроем файл **index.html** и изменим строку при отображении названий категорий:

```
<p class="first">Категория: {{p.cat.name}} | автор:
{{p.author.username|default:"неизвестен"}}</p>
```

Теперь на странице дополнительно отображается ее автор, если он известен.

Регистрация пользователей через функции представления

Познакомимся с регистрацией пользователей на сайте. У нас уже есть ссылка «Регистрация» на панели главного меню, давайте создадим для нее маршрут, функцию представления и шаблон по аналогии с тем, как мы это делали для задачи авторизации.

Сначала опишем шаблон формы регистрации в файле **users/register.html** следующим образом:

```
{% extends 'base.html' %}

{% block content %}
<h1>Регистрация</h1>

<form method="post">
    {% csrf_token %}
```

```

<input type="hidden" name="next" value="{{ next }}" />
    {{ form.as_p }}
    <p><button type="submit">Регистрация</button></p>
</form>

{% endblock %}

```

Здесь ничего нового для нас нет, мы это уже делали, когда говорили об авторизации. Объявим класс формы регистрации в файле **users/forms.py**:

```

class RegisterUserForm(forms.ModelForm):
    username = forms.CharField(label="Логин")
    password = forms.CharField(label="Пароль",
widget=forms.PasswordInput)
    password2 = forms.CharField(label="Повтор пароля",
widget=forms.PasswordInput)

    class Meta:
        model = get_user_model()
        fields = ['username', 'email', 'first_name',
'last_name', 'password', 'password2']
        labels = {
            'email': 'E-mail',
            'first_name': 'Имя',
            'last_name': 'Фамилия',
        }

```

Откуда мы знаем, какие поля следует прописывать в форме регистрации? Есть документация:

<https://docs.djangoproject.com/en/4.2/topics/auth/default/>

где подробно описываются стандартные поля модели **User**.

Форма у нас определена. Теперь можно объявить функцию представления в файле **users/views.py** следующим образом:

```

def register(request):
    form = RegisterUserForm()
    return render(request, 'users/register.html',
{'form': form})

```

Свяжем ее с маршрутом (в файле **users/urls.py**):

```

urlpatterns = [

```

```

    path('login/', views.LoginUser.as_view(),
name='login'),
    path('logout/', LogoutView.as_view(),
name='logout'),
    path('register/', views.register, name='register'),
]

```

Осталось прописать этот маршрут для ссылки «Регистрация». Откроем файл **base.html** и внесем следующие изменения:

```

<li class="last"><a href="{% url 'users:login'
%}">Войти</a> | <a href="{% url 'users:register'
%}">Регистрация</a></li>

```

Проверим, что получилось. Запустим веб-сервер и перейдем по ссылке «Регистрация». Видим отображение формы регистрации.

Проверка данных формы

Внесем в класс формы дополнение – метод **clean_password2()**, который будет проверять равенство введенных паролей. Напомним, что все методы формы, которые начинаются с префикса **clean_** и продолжаются именем поля, автоматически вызываются при проверке корректности переданных данных. Метод реализуем следующим образом:

```

def clean_password2(self):
    cd = self.cleaned_data
    if cd['password'] != cd['password2']:
        raise forms.ValidationError("Пароли не
совпадают!")
    return cd['password2']

```

Пропишем еще одну проверку на уникальность введенного E-mail адреса. В таблице **user** это поле не помечено, как уникальное, поэтому разные пользователи могут вводить один и тот же E-mail. Дополнительной проверкой мы это исключим:

```

def clean_email(self):
    email = self.cleaned_data['email']
    if User.objects.filter(email=email).exists():
        raise forms.ValidationError("Такой E-mail
уже существует!")
    return email

```

Теперь при попытке ввести повторяющийся E-mail адрес в форме будет отображаться ошибка.

Реализация процесса регистрации пользователя

Теперь воспользуемся этой формой и реализуем функционал для регистрации новых пользователей. Перейдем в файл **users/views.py** и перепишем функцию **register()** следующим образом:

```
def register(request):
    if request.method == "POST":
        form = RegisterUserForm(request.POST)
        if form.is_valid():
            user = form.save(commit=False)  # создание
            объекта без сохранения в БД
            user.set_password(form.cleaned_data['password'])
            user.save()
            return render(request,
                          'users/register_done.html')
        else:
            form = RegisterUserForm()
            return render(request, 'users/register.html',
                          {'form': form})
```

В целом, здесь все похоже на то, что мы делали при авторизации. Добавилось только формирование объекта пользователя с помощью метода **save()** формы. Причем с параметром **commit=False**, который запрещает запись данных непосредственно в БД. Мы это делаем позже, после установки пароля с помощью метода **set_password()**. Зачем понадобилось использовать этот метод? Дело в том, что пароли в таблице БД хранятся в зашифрованном виде. Там хранится только их хэш – закодированная определенным образом последовательность символов. Раскодировать хэш невозможно: алгоритмы хэширования работают в одну сторону – кодирования. Обратной операции они не подразумевают. Благодаря этому, злоумышленник, которому удастся скачать БД, не сможет получить доступ к аккаунтам пользователей, так как у него не будет паролей. Но тогда, как сам фреймворк Django сможет сравнить переданный ему от пользователя пароль, если в таблице хранится только его хэш? Django снова по тому же алгоритму захэширует переданный пароль и будет сравнивать их хэши. Если хэши совпадут, то делается вывод о корректности переданного пароля. Результат работы алгоритма хэширования уникальн для каждого проекта из-за секретного ключа **SECRET_KEY**, который прописан в файле **settings.py**. Этот ключ используется для внесения дополнительных искажений в результирующий хэш. Поэтому злоумышленнику еще нужно знать и эту дополнительную информацию.

Конечно, при большом желании взламываются многие системы. Есть приемы преодоления и этих преград защиты. Но сделать это будет весьма

непросто. Поэтому для большинства рядовых сайтов, включая и интернет-магазины, такого механизма хранения паролей вполне достаточно.

Итак, возвращаясь к нашей функции представления, нам нужно определить еще один шаблон с именем **register_done.html**:

```
{% extends 'base.html' %}

{% block content %}
<h1>Добро пожаловать!</h1>

<p>Вы успешно зарегистрировались на сайте! Для входа в
систему необходимо авторизоваться по этой <a href="{%
url 'users:login' %}">ссылке</a>
</p>
{% endblock %}
```

Регистрация пользователя в самом простом варианте реализована.

Класс UserCreationForm

Перейдем от функции представления **register()** к классу представления, а также воспользуемся стандартным классом **UserCreationForm** для создания нового пользователя.

Начнем с класса **UserCreationForm**. Перейдем в файл **users/forms.py** и переопределим класс **RegisterUserForm** следующим образом:

```
class RegisterUserForm(UserCreationForm):
    username = forms.CharField(label='Логин',
    widget=forms.TextInput(attrs={'class': 'form-input'}))
    password1 = forms.CharField(label='Пароль',
    widget=forms.PasswordInput(attrs={'class': 'form-
input'}))
    password2 = forms.CharField(label='Повтор пароля',
    widget=forms.PasswordInput(attrs={'class': 'form-
input'}))

    class Meta:
        model = get_user_model()
        fields = ['username', 'email', 'first_name',
'last_name', 'password1', 'password2']
        labels = {
            'email': 'E-mail',
            'first_name': 'Имя',
            'last_name': 'Фамилия',
```

```

    }
    widgets = {
        'email': forms.TextInput(attrs={'class':
'form-input'}),
        'first_name':
forms.TextInput(attrs={'class': 'form-input'}),
        'last_name':
forms.TextInput(attrs={'class': 'form-input'}),
    }

    def clean_email(self):
        email = self.cleaned_data['email']
        if User.objects.filter(email=email).exists():
            raise forms.ValidationError("Такой E-mail
уже существует!")
        return email

```

Здесь фигурируют атрибуты **password1** и **password2**. Это требование класса **UserCreationForm**:

<https://docs.djangoproject.com/en/4.2/topics/auth/customizing/#custom-users-and-the-built-in-auth-forms>

До этого у нас были атрибуты **password** и **password2**. Также здесь не нужно делать проверку на совпадение паролей. Оставим только фильтр на уникальность E-mail. Все остальное класс **UserCreationForm** берет на себя.

После определения класса перейдем в шаблон формы **register.html** и улучшим внешний вид формы:

```

<form method="post">
    {% csrf_token %}
    <input type="hidden" name="next" value="{{ next }}"
/>
    <div class="form-error">{{ form.non_field_errors
}}</div>
    {% for f in form %}
    <p ><label class="form-label" for="{{
f.id_for_label }}">{{f.label}}: </label>{{ f }}</p>
    <div class="form-error">{{ f.errors }}</div>
    {% endfor %}
    <p ><button type="submit">Регистрация</button></p>
</form>

```

Теперь форма регистрации выглядит лучше. Последнее, что осталось сделать – это заменить функцию представления **register** на класс.

Специального класса для регистрации в Django нет, в нем и нет смысла, так как все, что нам нужно сделать – это проверить данные формы и сохранить ее в БД. Для этой операции уже есть класс представления **CreateView**. Воспользуемся им и в файле **users/views.py** объявим класс **RegisterUser** следующим образом:

```
class RegisterUser(CreateView):
    form_class = RegisterUserForm
    template_name = 'users/register.html'
    extra_context = {'title': "Регистрация"}
```

И свяжем этот класс с маршрутом **register** в файле **users/urls.py**:

```
path('register/', views.RegisterUser.as_view(),
name='register'),
```

Обновим страницу регистрации, введем существующий логин, email, несовпадающие пароли и увидим сгенерированные сообщения об ошибках, которые автоматически создает форма **UserCreationForm**. И в этом ее удобство. Нам не нужно здесь выполнять каждый раз типовую работу. Фреймворк Django все берет на себя. От нас требуется только использовать специально предназначенные классы для той или иной задачи.

Заполним форму корректными данными, нажмем на кнопку «Регистрация» и получаем ошибку, что в модели не определен метод **get_absolute_url()**. Он используется, если Django не знает, куда перенаправить пользователя. Исправим это, добавив в класс **RegisterUser** известный нам атрибут:

```
success_url = reverse_lazy('users:login')
```

Теперь при регистрации мы сразу будем попадать на страницу авторизации. А в админ-панели появился еще один пользователь.

Авторизация через email

Сейчас реализуем довольно распространенный способ авторизации по E-mail адресу и паролю. Для этого при регистрации пользователей мы требовали уникальный E-mail. Вообще, им можно заменить логин и не просить пользователя вводить эти дополнительные данные.

По умолчанию в Django используется бэкенд **ModelBackend** для аутентификации пользователя по паре логин (username) и пароль (password):

<https://docs.djangoproject.com/en/4.2/topics/auth/customizing/>

В настроечном файле **settings.py** его можно явно прописать, используя параметр **AUTHENTICATION_BACKENDS**:

```
AUTHENTICATION_BACKENDS = [  
    'django.contrib.auth.backends.ModelBackend',  
]
```

Если перейти к определению класса **ModelBackend**, то увидим, что он наследуется от базового класса **BaseBackend**, в котором определены методы, необходимые для работы механизма аутентификации. При этом главными, непосредственно для аутентификации пользователя, являются методы:

- **authenticate()** – непосредственно аутентификация по **username** и **password**; возвращается объект пользователя, либо **None**, если он не был найден;
- **get_user()** – получение объекта пользователя по идентификатору.

Чтобы создать свой бэкенд аутентификации нужно, как минимум, определить в нем эти два метода. Сделаем это. Для этого в приложении **users** создадим еще один файл с именем **authentication.py**, в котором определим новый класс бэкенда с именем **EmailAuthBackend** следующим образом:

```
from django.contrib.auth.backends import BaseBackend  
  
class EmailAuthBackend(BaseBackend):  
    ...
```

И добавим в этот класс метод **authenticate()**, сохранив его сигнатуру (наборы параметров), следующим образом:

```
class EmailAuthBackend(BaseBackend):  
    def authenticate(self, request, username=None,  
password=None, **kwargs):  
        user_model = get_user_model()  
        try:  
            user =  
user_model.objects.get(email=username)  
            if user.check_password(password):  
                return user  
            return None  
        except (user_model.DoesNotExist,  
user_model.MultipleObjectsReturned):  
            return None
```


Здесь используется стандартное имя параметра **username**, чтобы наш новый бэкенд согласованно работал с фреймворком Django. Но мы подразумеваем, что сюда будет передаваться E-mail, по которому, затем, выделяется запись из таблицы **user**. Если запись найдена и пароль совпадает, то аутентификация прошла успешно и возвращается объект пользователя. Иначе возвращаем **None**, а также в том случае, если запись не была найдена или было получено несколько записей с указанным E-mail.

Мы уже сейчас можем подключить наш бэкенд к Django и посмотреть, как он будет работать. Для этого в файле конфигурации **settings.py** в список **AUTHENTICATION_BACKENDS** следует добавить строку:

```
AUTHENTICATION_BACKENDS = [  
    'django.contrib.auth.backends.ModelBackend',  
    'users.authentication.EmailAuthBackend',  
]
```

У нас будут работать оба бэкенда: и по логину и по E-mail. Эти классы просматриваются по порядку в списке и срабатывает первый вернувший объект пользователя, остальные пропускаются. В результате пользователи могут авторизоваться и по логину и по E-mail.

Если сейчас запустить тестовый веб-сервер и перейти в форму авторизации, то при вводе корректного E-mail и пароля мы перейдем на главную страницу. Это результат работы метода **authenticate()**. Но при этом, в главном меню не видим отображения имени пользователя. Это из-за того, что мы не прописали второй метод **get_user()** в классе **EmailAuthBackend**:

```
def get_user(self, user_id):  
    user_model = get_user_model()  
    try:  
        return user_model.objects.get(pk=user_id)  
    except user_model.DoesNotExist:  
        return None
```

Теперь при вводе E-mail и пароля пользователь будет авторизован и отображен на панели главного меню.

Создание профайла пользователя

Создадим страницу с профилем пользователя. Сначала определим шаблон в файле **users/profile.html** следующим образом:

```
{% extends 'base.html' %}  
  
{% block content %}
```

```

<h1>Профиль</h1>

<form method="post">
    {% csrf_token %}
    <div class="form-error">{{ form.non_field_errors }}</div>
    {% for f in form %}
    <p ><label class="form-label" for="{{ f.id_for_label }}">{{f.label}}: </label>{{ f }}</p>
    <div class="form-error">{{ f.errors }}</div>
    {% endfor %}

    <p ><button type="submit">Сохранить</button></p>
</form>

{% endblock %}

```

И класс формы **ProfileUserForm** для этого шаблона:

```

class ProfileUserForm(forms.ModelForm):
    username = forms.CharField(disabled=True,
label='ЛОГИН', widget=forms.TextInput(attrs={'class':
'form-input'}))
    email = forms.CharField(disabled=True, label='E-
mail', widget=forms.TextInput(attrs={'class': 'form-
input'}))

    class Meta:
        model = get_user_model()
        fields = ['username', 'email', 'first_name',
'last_name']
        labels = {
            'first_name': 'Имя',
            'last_name': 'Фамилия',
        }
        widgets = {
            'first_name':
forms.TextInput(attrs={'class': 'form-input'}),
            'last_name':
forms.TextInput(attrs={'class': 'form-input'}),
        }

```

У полей **username** и **email** мы установили параметр **disabled=True**, сделали их неактивными и неизменяемыми. Все остальное можно редактировать.

В файле **users/view.py** объявим класс представления для работы с этой формой:

```
class ProfileUser(LoginRequiredMixin, UpdateView):
    model = get_user_model()
    form_class = ProfileUserForm
    template_name = 'users/profile.html'
    extra_context = {'title': "Профиль пользователя"}

    def get_success_url(self):
        return reverse_lazy('users:profile',
args=[self.request.user.pk])
```

Мы его наследуем от класса **LoginRequiredMixin**, чтобы запретить доступ неавторизованным пользователям. А класс **UpdateView** берет на себя функционал по изменению данных в профиле пользователя. Метод **get_success_url()** необходим для перенаправления на текущую страницу при изменении данных.

Свяжем класс **ProfileUser** с маршрутом (в файле **users/urls.py**):

```
path('profile/<int:pk>/', views.ProfileUser.as_view(),
name='profile'),
```

Обратите внимание на обязательный параметр **pk**, по которому выбирается запись из БД по текущему пользователю.

Однако описанный маршрут позволяет просматривать и редактировать любые профайлы, достаточно указать идентификатор пользователя, и вся информация о нем будет отображена на странице. Поэтому мы уберем отбор записей из таблицы **user** по идентификатору:

```
path('profile/', views.ProfileUser.as_view(),
name='profile'),
```

а в классе представления **ProfileUser** добавим следующий метод:

```
def get_object(self, queryset=None):
    return self.request.user
```

Теперь профайл будет открываться только для текущего пользователя, либо будет сделано перенаправление на страницу авторизации для неавторизованных пользователей.

Пропишем маршрут к профилю в главном меню. Для этого перейдем в шаблон **base.html** и внесем следующие изменения:

```
<li class="last"> <a href="{% url 'users:profile' %}">{{user.username}}</a> | <a href="{% url 'users:logout' %}">Выйти</a></li>
```

Классы PasswordChangeView и PasswordChangeDoneView

Мы создали профиль пользователя. Но в нем не хватает одного важного пункта – возможности смены пароля.

Фреймворк Django предоставляет два следующих класса для изменения пароля пользователя:

- **PasswordChangeView** – для обработки формы изменения пароля;
- **PasswordChangeDoneView** – для отображения результата успешного изменения пароля.

Если мы посмотрим объявление класса **PasswordChangeView**, то увидим, что он использует по умолчанию шаблон **registration/password_change_form.html** и класс формы **PasswordChangeForm**, а также делает перенаправление по маршруту с именем **password_change_done**. Мы можем воспользоваться всеми этими стандартными настройками, просто прописав в файле **users/urls.py** соответствующие маршруты:

```
urlpatterns = [  
    path('login/', views.LoginUser.as_view(),  
name='login'),  
    path('logout/', LogoutView.as_view(),  
name='logout'),  
    path('password-change/',  
PasswordChangeView.as_view(), name='password_change'),  
    path('password-change/done/',  
PasswordChangeDoneView.as_view(),  
name='password_change_done'),  
  
    path('register/', views.RegisterUser.as_view(),  
name='register'),  
    path('profile/<int:pk>/',  
views.ProfileUser.as_view(), name='profile'),  
]
```

Добавим в шаблон **profile.html** ссылку для изменения пароля:

```
<hr>  
<p><a href="{% url 'users:password_change' %}">Сменить  
пароль</a></p>
```

После запуска веб-сервера и перехода в профиль пользователя, увидим ссылку «Сменить пароль». Если сейчас перейти по ней, то окажемся в админ-панели со стандартной формой смены пароля.

Но мы сделаем свою форму. Для этого создадим файл шаблона **users/password_change_form.html** со стандартным содержимым:

```
{% extends 'base.html' %}

{% block content %}
<h1>Изменение пароля</h1>

<form method="post">
    {% csrf_token %}
    <div class="form-error">{{ form.non_field_errors }}</div>
    {% for f in form %}
    <p ><label class="form-label" for="{{ f.id_for_label }}">{{f.label}}: </label>{{ f }}</p>
    <div class="form-error">{{ f.errors }}</div>
    {% endfor %}

    <p ><button type="submit">Изменить
пароль</button></p>
</form>

{% endblock %}
```

Здесь стандартное содержимое для отображения полей формы в нашем базовом шаблоне.

Затем для этого шаблона определим класс формы на базе стандартного класса **PasswordChangeForm** и настроим в нем отображение следующих полей:

```
class UserPasswordChangeForm(PasswordChangeForm):
    old_password = forms.CharField(label="Старый
пароль", widget=forms.PasswordInput(attrs={'class':
'form-input'}))
    new_password1 = forms.CharField(label="Новый
пароль", widget=forms.PasswordInput(attrs={'class':
'form-input'}))
    new_password2 =
forms.CharField(label="Подтверждение пароля",
widget=forms.PasswordInput(attrs={'class': 'form-
input'}))
```

Набор этих полей можно посмотреть в классе **PasswordChangeForm**.

Добавим свой класс представления для отображения собственной формы:

```
class UserPasswordChange (PasswordChangeView) :
    form_class = UserPasswordChangeForm
    success_url =
reverse_lazy("users:password_change_done")
    template_name = "users/password_change_form.html"
    extra_context = {'title': "Изменение пароля"}
```

И подключим его к маршруту **password_change**:

```
path('password-change/',
views.UserPasswordChange.as_view(),
name='password_change'),
```

Если теперь перейти по ссылке «Изменить пароль», то увидим форму в стиле оформления нашего сайта. Причем эта форма полностью рабочая. Но нам осталось еще прописать свой собственный шаблон для маршрута **password_change_done** с информацией об успешном изменении пароля. Для этого мы создадим еще один шаблон **users/password_change_done.html** со следующим содержимым:

```
{% extends 'base.html' %}

{% block content %}
<h1>Пароль успешно изменен!</h1>

<p>Вы успешно изменили пароль. <a href="{% url
'users:profile' %}">Вернуться в профиль.</a></p>
{% endblock %}
```

И пропишем его в методе **as_view()** класса **PasswordChangeDoneView**:

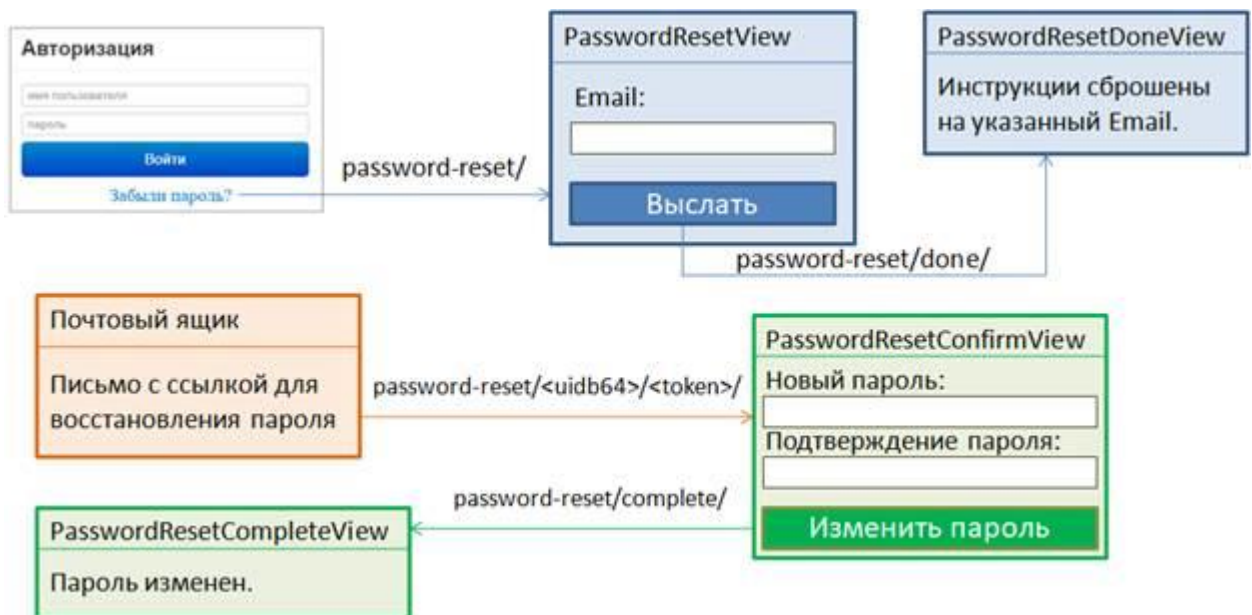
```
path('password-change/done/',
PasswordChangeDoneView.as_view(template_name="users/pas
sword_change_done.html"), name="password_change_done"),
```

После изменения попадаем на страницу об успешном изменении пароля.

Восстановление пароля

Все мы неоднократно видели на формах авторизации ссылку с вопросом «Забыли пароль?» и, скорее всего, даже пользовались ей. Она

позволяет нам создать новый пароль для своего аккаунта в случаях, когда мы его забываем. Это не редкая ситуация, поэтому на любом серьезном сайте должна быть возможность восстановления пароля и сейчас мы познакомимся с идеей этого механизма во фреймворке Django.



При нажатии на ссылку «Забыли пароль?» по маршруту **password-reset** открывается страница с указанием E-mail, связанного с аккаунтом пользователя. При нажатии на кнопку «Выслать» на электронный ящик отправляется письмо со ссылкой для восстановления пароля и осуществляется переход по маршруту **password-reset/done/** с отображением информационного сообщения. Затем в почтовом ящике пользователь должен открыть письмо и перейти по указанной одноразовой ссылке. Появляется страница с установкой нового пароля, который меняется при нажатии на кнопку «Изменить пароль». После этого автоматически осуществляется переход по адресу **password-reset/complete** с отображением информационного сообщения успешного изменения пароля.

Такая последовательность действий выполняется для смены пароля. Здесь предполагается, что, если пользователь помнит свой E-mail адрес, связанный с аккаунтом и пароль доступа к ящику, то это тот самый пользователь и ему можно предоставить возможность для восстановления пароля.

Конечно, эта схема не гарантирует полностью от взлома аккаунта со стороны злоумышленника. Существуют приемы для получения пароля от аккаунта, если известен пароль от почтового ящика. Но, тем не менее, она неплохо себя зарекомендовала за долгие годы применения в самых разных приложениях. В подавляющем большинстве случаев операция восстановления пароля себя оправдывает, и следующим шагом мы добавим этот функционал на наш сайт.

Настройка консольного почтового бэкенда

В схеме восстановления пароля нужно иметь возможность отправлять письма пользователю с почтового ящика нашего сервиса (сайта). Обычно почтовый сервер разворачивают совместно с обычным сервером, который обрабатывает скрипты сайта. Хотя для эксперимента можно воспользоваться любым известным сторонним почтовым сервером, например, от компании Яндекс или Google. Но серьезный проект подразумевает использование собственного почтового сервера, так как в этом случае мы получаем полный контроль над его работой.

Но пока в процессе разработки сайта мы используем тестовый веб-сервер и для задач тестирования приложения настроим фреймворк Django на отправку писем в консоль. На странице документации:

<https://docs.djangoproject.com/en/4.2/topics/email/>

подробно описаны функции отправки писем. А также приведены настройки различных почтовых бэкендов. В частности по ссылке:

<https://docs.djangoproject.com/en/4.2/topics/email/#console-backend>

мы видим описание параметра **EMAIL_BACKEND** для настройки консольного бэкенда. Для этого в файле настроек **settings.py** следует прописать этот параметр со значением:

```
EMAIL_BACKEND =  
"django.core.mail.backends.console.EmailBackend"
```

По умолчанию используется SMTP backend и приведен набор его настроек для реального почтового сервера, работающего по протоколу SMTP. Позже мы рассмотрим пример его использования. А пока оставим консольный бэкенд.

Посмотрим, как он будет работать. Войдем в оболочку shell:

```
python manage.py shell
```

И с самого начала страницы документации «Sending email» скопируем строку импортирования функции **send_mail()**:

```
from django.core.mail import send_mail
```

Затем скопируем пример отправки письма этой функцией:

```
send_mail(  
    "От Иванова",
```



```
"Поздравляем! Вы успешно изучили Django.",
"root@site.com",
["you@mail.com"],
)
```

Здесь же в консоли увидим примерно такое почтовое сообщение:

```
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
To: you@mail.com
Date: Sat, 18 May 2024 04:38:55 -0000
Message-ID: <168879113585.16272.11887720130122423793@COMP-135 >
```

Поздравляем! Вы успешно изучили Django.

Консольный почтовый бэкенд успешно подключен и работает.

Реализация алгоритма восстановления пароля

Приступим непосредственно к реализации механизма восстановления паролей в соответствии с приведенной выше схемой. Сначала создадим шаблон формы **password_reset_form.html** для ввода E-mail адреса:

```
{% extends 'base.html' %}

{% block content %}
<h1>Восстановление пароля</h1>

<form method="post">
    {% csrf_token %}
    {% for f in form %}
    <p ><label class="form-label" for="{{ f.id_for_label }}">{{ f.label }}: </label>{{ f }}</p>
    <div class="form-error">{{ f.errors }}</div>
    {% endfor %}

    <p ><button type="submit">Сбросить по E-mail</button></p>
</form>

{% endblock %}
```

И шаблон **password_reset_done.html** с информацией о сбросе пароля по E-mail (скопирован из **register/password_reset_done.html** и немного изменен):

```
{% extends "base.html" %}
```

```
{% load i18n %}

{% block content %}
<h1>Сброс пароля</h1>
<p >{% translate 'We've emailed you instructions for
setting your password, if an account exists with the
email you entered. You should receive them shortly.'
%}</p>
<p >{% translate 'If you don't receive an email, please
make sure you've entered the address you registered
with, and check your spam folder.' %}</p>
{% endblock %}
```

Воспользуемся стандартными классами представлений **PasswordResetView** и **PasswordResetDoneView**, у которых укажем пути к нашим шаблонам. В файле **users/urls.py** пропишем следующие строки:

```
path('password-reset/',
     PasswordResetView.as_view(template_name =
"users/password_reset_form.html"),
     name='password_reset'),
path('password-reset/done/',
     PasswordResetDoneView.as_view(template_name =
"users/password_reset_done.html"),
     name='password_reset_done'),
```

Добавим в шаблон формы авторизации ссылку для восстановления пароля (в файл **login.html**):

```
<p ><a href="{% url 'users:password_reset' %}">Забыли
пароль?</a></p>
```

Запустим веб-сервер, откроем окно авторизации и нажмем на ссылку «Забыли пароль?». Появляется форма ввода E-mail-адреса пользователя. Вводим E-mail и нажимаем на кнопку «Отправить по E-mail». После чего у нас выскакивает ошибка, что маршрут с именем **password_reset_confirm** не определен. Действительно, мы реализовали только первую часть схемы. Давайте пропишем и вторую.

Сформируем еще два шаблона:

- файл **password_reset_confirm.html**:

```
{% extends 'base.html' %}

{% block content %}
```

```

<h1>Новый пароль</h1>

<form method="post">
    {% csrf_token %}
    <div class="form-error">{{ form.non_field_errors }}</div>
    {% for f in form %}
    <p ><label class="form-label" for="{{ f.id_for_label }}">{{f.label}}: </label>{{ f }}</p>
    <div class="form-error">{{ f.errors }}</div>
    {% endfor %}

    <p ><button type="submit">Сохранить</button></p>
</form>

{% endblock %}

```

- файл **password_reset_complete.html**:

```

{% extends 'base.html' %}

{% block content %}
<h1>Пароль изменен</h1>

<p >Поздравляем! Вы успешно изменили пароль своего
аккаунта. Используйте его для
    <a href="{% url 'users:login' %}">входа в
систему</a>.</p>

{% endblock %}

```

И в файле **users/urls.py**, воспользуемся стандартными классами представлений **PasswordResetConfirmView** и **PasswordResetCompleteView**:

```

path('password-reset/<uidb64>/<token>/',
PasswordResetConfirmView.as_view(template_name="users/p
assword_reset_confirm.html"),
name='password_reset_confirm'),
path('password-reset/complete/',
PasswordResetCompleteView.as_view(template_name="users/
password_reset_complete.html"),
name='password_reset_complete'),

```

Однако, если сейчас попробовать восстановить пароль, то получим ошибку:

NoReverseMatch at /users/password-reset/

Она произошла из-за того, что в стандартном шаблоне **registration/password_reset_email.html** используется тег:

```
{% url 'password_reset_confirm' uidb64=uid token=token %}
```

без указания пространства имен **users**. Поэтому скопируем этот шаблон и подправим в нем этот тег:

```
{{ protocol }}://{{ domain }}{% url  
'users:password_reset_confirm' uidb64=uid token=token  
%}
```

И в файле **users/urls.py** пропишем этот новый шаблон для класса представления **PasswordResetView** следующим образом:

```
path('password-reset/',  
     PasswordResetView.as_view(  
  
template_name="users/password_reset_form.html",  
email_template_name="users/password_reset_email.html",  
success_url=reverse_lazy("users:password_reset_done")  
),  
     name='password_reset'),
```

Мы здесь добавили параметр **success_url** с указанием пространства имен **users**. Иначе, при перенаправлении возникла бы та же ошибка. Теперь все проходит, мы видим письмо со ссылкой для восстановления пароля и информационную страницу о его сбросе на почтовый ящик.

Прежде чем переходить по этой ссылке, мы пропишем параметр **success_url** и у класса представления **PasswordResetConfirmView**:

```
path('password-reset/<uidb64>/<token>/',  
     PasswordResetConfirmView.as_view(  
  
template_name="users/password_reset_confirm.html",  
success_url=reverse_lazy("users:password_reset_complete")  
),  
     name='password_reset_confirm'),
```

Значение параметра **success_url** можно посмотреть в самом классе **PasswordResetConfirmView**.

Снова сделаем сброс пароля, переходим по ссылке в письме и попадаем на страницу изменения пароля. Вводим его, и он сохраняется для пользователя с указанным E-mail адресом.

Настройка почтового сервера по SMTP-протоколу

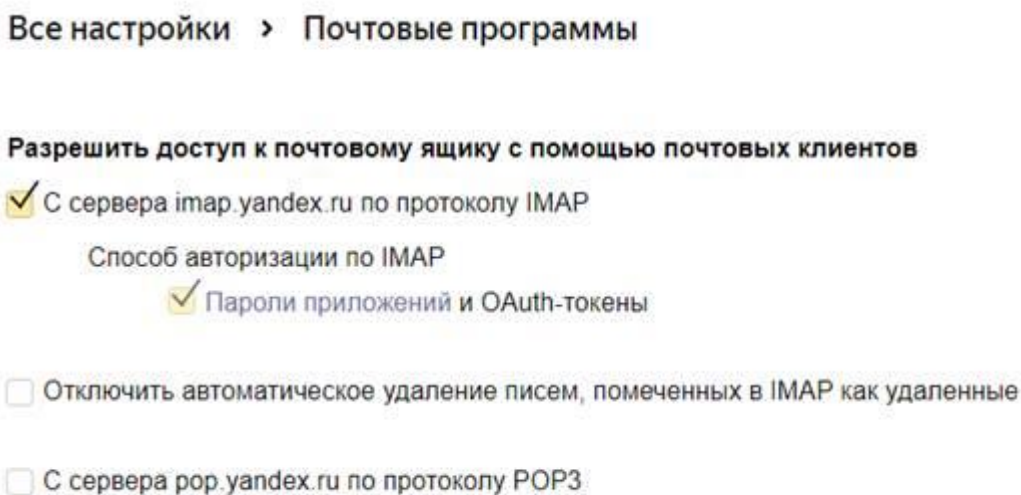
Теперь воспользуемся реальным SMTP почтовым сервером для отправки сообщений о восстановлении пароля пользователям. В качестве примера рассмотрим использование почтового сервера компании Яндекс. Но принцип будет тем же и для других SMTP-серверов.

Предположим, что мы зарегистрировали аккаунт в Яндексе по ссылке:

passport.yandex.ru/registration/

с почтовым ящиком `django@yandex.ru`. Настроим сервер для этого ящика с возможностью отправлять почтовые сообщения из фреймворка Django.

В аккаунте почты Яндекса перейдем в расширенные настройки и далее в «Почтовые программы». Ставим галочки в разделе «Разрешить доступ к почтовому ящику с помощью почтовых клиентов»:



Чтобы не столкнуться с ошибкой аутентификации и правильно настроить SMTP для Yandex, нам необходимо сгенерировать пароль для приложения, чтобы мы смогли аутентифицироваться из нашего проекта на Django. Для этого переходим в свой Яндекс ID («Управление аккаунтом»), далее «Безопасность», затем «Доступы» и «Пароли приложений»:

<https://id.yandex.ru/security/app-passwords>

На этой странице выбрать «Почта» и ввести любой пароль, можно от текущего почтового ящика. Система автоматически сгенерирует пароль для приложения и его нужно обязательно скопировать, так как он показывается только один раз. Скопированный пароль сразу же сохраним в файле **settings.py** нашего проекта Django в параметре:

```
EMAIL_HOST_PASSWORD = "zbuekakprdsqnleg"
```

После этого в этом же файле укажем использовать почтовый SMTP-бэкенд. Можно убрать параметр **EMAIL_BACKEND** или прописать его со значением:

```
EMAIL_BACKEND =  
"django.core.mail.backends.smtp.EmailBackend"
```

Подробно о различных бэкендах фреймворка Django можно посмотреть на странице:

<https://docs.djangoproject.com/en/4.2/topics/email/#email-backends>

Также здесь видим набор параметров, которые необходимо дополнительно прописать для настройки SMTP-бэкенда, то есть привязки его к нашему почтовому ящику.

Чтобы найти данные для этих параметров, заходим в официальную документацию Яндекса:

<https://yandex.ru/support/mail/mail-clients/others.html>

и здесь на шаге 3 видим значения необходимых настроек для входящих писем и для исходящих. Нам нужно для исходящих, так как мы будем отправлять письма с нашего почтового ящика.

В нашем случае настройки будут следующими:

```
EMAIL_HOST = "smtp.yandex.ru"  
EMAIL_PORT = 465  
EMAIL_HOST_USER = "django@yandex.ru"  
EMAIL_HOST_PASSWORD = "bnufhkwcripaunvu"  
EMAIL_USE_SSL = True
```

Кроме того, для отправки сообщений от имени приложения необходимо добавить еще такие параметры:

```
DEFAULT_FROM_EMAIL = EMAIL_HOST_USER  
SERVER_EMAIL = EMAIL_HOST_USER  
EMAIL_ADMIN = EMAIL_HOST_USER
```

Конечно, вместо EMAIL_HOST_USER можно указать любой другой E-mail адрес. Главное, чтобы эти три параметра также были прописаны.

На этом настройка SMTP-бэкенда во фреймворке Django завершена, и он теперь связан с почтовым сервером Яндекса. Проверим, как все будет работать. Перейдем на страницу восстановления пароля:

<http://127.0.0.1:8000/users/password-reset/>

Вводим E-mail адрес, который присутствует в БД (это обязательно, иначе письмо не будет отправлено). И нажимаем на кнопку «Сбросить пароль». Если все было сделано правильно, то должна отобразиться информационная страница со сбросом пароля. Переходим в почтовый ящик, куда письмо было отправлено и видим в нем новое письмо со ссылкой для восстановления пароля. Копируем ссылку, вставляем в адресную строку браузера и появляется форма изменения пароля. Создаем новый пароль, и он успешно меняется для текущего пользователя. Теперь мы можем войти в аккаунт, используя E-mail адрес и пароль, который только что создали. Все работает.

Расширение модели User

Если мы посмотрим на таблицу auth_user, то набор информационных полей будет примерно таким:

- **username** – логин пользователя;
- **password** – пароль для входа в систему;
- **email** – электронная почта;
- **first_name** – имя пользователя;
- **last_name** – фамилия пользователя.

Но, что, если нам дополнительно нужно хранить фотографию пользователя, его дату рождения, пол, город, семейное положение и так далее. Как это сделать? Как расширить уже существующую модель User? Во фреймворке Django для этой цели используют два разных подхода:

- создание еще одной модели (например, **Profile**) со связью one-to-one (один к одному) с моделью **User**;
- создание новой модели **User** на базе специального класса **AbstractUser** фреймворка Django.

У каждого подхода есть свои преимущества и недостатки. Преимущества первого состоят в использовании стандартной модели **User**, а, значит, все модули, которые напрямую обращаются к ней, продолжат работать без каких-либо изменений. А основной недостаток – это необходимость в запросах отдельно указывать связанную модель **Profile**. Преимущества второго подхода в том, что мы, как и ранее, продолжаем

работать с одной моделью пользователя, а, значит, скорость обработки запросов, в среднем, несколько возрастает. И, кроме того, нам не нужно помнить о дополнительных моделях, что удобно. Главный недостаток – это уход от стандартной модели, поэтому нужно быть уверенным, что во всех компонентах проекта фреймворка Django обращение идет именно к этой новой таблице.

На практике используют оба подхода в зависимости от удобства разработки текущего проекта. Здесь мы рассмотрим второй подход - расширение стандартной модели с помощью базового класса `AbstractUser`, поскольку так мы получаем более нативное решение и, по-прежнему, привычно работаем с одной моделью.

Класс `AbstractUser`

Согласно документации Django:

<https://docs.djangoproject.com/en/4.2/topics/auth/customizing/#extending-the-existing-user-model>

мы можем расширить существующую модель `User` следующим образом (в файле `users/models.py`):

```
class User(AbstractUser):
    photo =
models.ImageField(upload_to="users/%Y/%m/%d/",
blank=True, null=True, verbose_name="Фотография")
    date_birth = models.DateTimeField(blank=True,
null=True, verbose_name="Дата рождения")
```

Также в документации сказано, что в файле `users/admin.py` необходимо зарегистрировать эту новую модель для админ-панели следующим образом:

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin

from users.models import User

admin.site.register(User, UserAdmin)
```

Наша модель создана и зарегистрирована. Причем в ней будут присутствовать все стандартные поля прежней модели `User`, благодаря наследованию от класса `AbstractUser`, и два дополнительных поля: `photo` и `date_birth`. Однако, если мы сейчас попробуем создать миграцию для создания новой таблицы `user`, то увидим ошибки. Это связано с тем, что при замене существующей модели `User` на свою собственную, нам в файле

`settings.py` необходимо еще переопределить параметр `AUTH_USER_MODEL`, который по умолчанию принимает значение `'auth.User'`. В нашем случае параметр `AUTH_USER_MODEL` следует прописать так:

```
AUTH_USER_MODEL = 'users.User'
```

Сначала указывается имя приложения, затем через точку имя используемой модели в текущем проекте фреймворка Django. Теперь при создании миграции:

```
python manage.py makemigrations
```

никаких ошибок не возникает. Но применение созданной миграции:

```
python manage.py migrate
```

снова приводит к ошибкам. Это из-за того, что прежняя модель **User** в БД связана с другими таблицами. Нам придется удалить все файлы миграций из приложений **users** и **women**, а также файл базы данных.

Такие радикальные шаги приходится делать, так как целиком замещается стандартная модель **User** на совершенно другую из приложения **users**. Чтобы таких неудобств не возникало, следует заранее продумывать вопрос о расширении стандартной модели. Тогда ничего удалять не придется. Причем число и тип полей в новой расширенной модели мы впоследствии сможем изменить совершенно спокойно без удаления файлов миграций. Только замена модели целиком приводит к переделке всей БД.

Итак, удаляем все миграции из приложений **users** и **women**, удаляем файл БД и снова выполняем команды:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Теперь ошибок нет, и все таблицы были созданы успешно.

Следующим шагом следует убрать во всем проекте обращение к стандартной модели **User**, если это имело место, и вместо нее прописать функцию `get_user_model()`.

Если при попытке запуска проекта возникает ошибка:

```
AUTH_USER_MODEL refers to model 'users.get_user_model()' that has not been installed
```

то в файле **settings.py** нужно проверить определение параметра **AUTH_USER_MODEL**, которое должно быть таким:

```
AUTH_USER_MODEL = 'users.User'
```

Теперь проект запустился с полным функционалом сайта, но без содержимого. Сначала нам нужно снова создать суперпользователя:

```
python manage.py createsuperuser
```

Введем прежние данные и пробуем войти в систему. Все работает, но уже с новой расширенной таблицей **User**. Пробуем зарегистрироваться, Нажимаем на кнопку «Регистрация» и попадаем на страницу авторизации. Попробуем авторизоваться по E-mail и паролю. Успешно входим в систему, все работает.

Вручную через админ-панель добавим данные в наши таблицы, чтобы сайт вновь приобрел свое содержимое.

Наконец, добавим отображение даты рождения и фотографию пользователя в его профиле. Сначала в файле **users/forms.py** в классе **ProfileUserForm** сделаем отображение даты рождения:

```
class ProfileUserForm(forms.ModelForm):
    ...
    this_year = datetime.date.today().year
    date_birth =
forms.DateField(widget=forms.SelectDateWidget(years=tuple(
range(this_year-100, this_year-5))))

    class Meta:
        model = get_user_model()
        fields = ['photo', 'username', 'email',
'date_birth', 'first_name', 'last_name']
    ...
```

Чтобы воспользоваться модулем **datetime**, сначала его следует импортировать:

```
import datetime
```

Осталось сделать отображение текущего изображения в профиле пользователя. Для этого перейдем в файл шаблона **users/profile.html** и добавим в него строки:

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
```

```
{% if user.photo %}
<p >
{% else %}
<p >
{% endif %}
...
```

Соответственно, в каталоге **media** в папке **users** следует поместить файл **default.png**.

Однако, явно прописывать путь к **default.png** не лучшая практика. Если путь изменится, придется править все строки, где он упоминается в шаблонах. Поэтому мы через параметр **extra_context** класса **ProfileUser** (в файле **users/views.py**) передадим этот маршрут следующим образом:

```
extra_context = {'title': "Профиль пользователя",
'default_image': settings.DEFAULT_USER_IMAGE}
```

А параметр **DEFAULT_USER_IMAGE** определим в файле **settings.py** нашего проекта:

```
DEFAULT_USER_IMAGE = MEDIA_URL + 'users/default.png'
```

Теперь в шаблоне **profile.html** достаточно прописать переменную **default_image**:

```
<p >
```

Разрешения и группы (Permissions & Groups)

Благодаря наследованию новой модели от класса **AbstractUser**, в ней содержатся все стандартные поля и внешние ключи прежней модели. Если открыть файл **users/migrations/0001_initial.py**, то увидим внешние ключи с именами:

- **groups** – группы, которым принадлежит пользователь;
- **user_permissions** – разрешения, которые имеются у пользователя.

Подробная документация о группах и разрешениях приведена на следующей странице:

<https://docs.djangoproject.com/en/4.2/topics/auth/default/#permissions-and-authorization>

Начнем с разрешений (permissions). Если открыть админ-панель сайта, то в разделе пользователей:

<http://127.0.0.1:8000/admin/users/user/>

при их редактировании в самом низу видим возможность назначения групп и разрешений. Причем список стандартных разрешений для разных таблиц БД уже сформирован. Фреймворк Django делает это автоматически. Как только появляется новая таблица, то для нее создается минимум четыре разрешения:

- **add** – на добавление новых записей;
- **change** – на изменение существующих записей;
- **delete** – на удаление записей;
- **view** – на просмотр записей.

Для примера какому-нибудь пользователю добавим разрешение на добавление записи в таблицу **women**. Сейчас это действие никак не влияет на функционал нашего сайта. По-прежнему, добавлять статью может любой пользователь. Чтобы, например, ограничить доступ к странице:

<http://127.0.0.1:8000/addpage/>

и отображать ее только пользователям с разрешением на добавление новой записи в таблицу **women**, в классе представления **AddPage** следует прописать базовый класс миксина **PermissionRequiredMixin**, а само разрешение указать с помощью атрибута **permission_required** следующим образом:

```
permission_required = 'women.add_women'
```

Обратите внимание на имя стандартного разрешения. Сначала пишется имя приложения, в котором срабатывает представление **AddPage**, затем, через точку название самого разрешения **add** и через подчеркивание – имя таблицы **women**. Так определяются имена всех стандартных разрешений:

<приложение>.<действие>_<таблица>

Теперь пользователь, которому мы ранее дали разрешение, получает доступ к странице «Добавить статью». Если попробовать зайти на эту страницу под другим пользователем, который не имеет этого разрешения, то увидим код ошибки 403 – доступ **запрещен**. Но суперпользователь по умолчанию имеет все права, поэтому он также может зайти на эту страницу.

Давайте еще ограничим доступ к странице редактирования постов. За это отвечает представление **UpdatePage**. Также унаследуем его от класса миксина **PermissionRequiredMixin** и пропишем атрибут:

```
permission_required = 'women.change_women'
```

Теперь ни один из пользователей (кроме суперпользователя) не может получить доступ к этой странице:

<http://127.0.0.1:8000/edit/shakira/>

Чтобы какой-нибудь пользователь получил доступ к странице редактирования, нужно дать ему право менять и удалять записи.

Если же мы хотим назначить разрешение функции представления, то это делается с помощью специального декоратора **permission_required**:

```
@permission_required(perm='women.view_women',
raise_exception=True)
def contact(request):
    return HttpResponse("Обратная связь")
```

Здесь второй параметр **raise_exception** нужен для генерации кода 403 – доступ запрещен. Иначе, мы будем перенаправлены на страницу авторизации, что было бы странно для авторизованного пользователя.

Использование разрешений в шаблонах

Следующим шагом посмотрим, как можно использовать разрешения внутри шаблонов. Предположим, мы бы хотели выводить ссылку «Редактировать» при просмотре статей, но только тем пользователям, которые имеют право менять существующие записи в таблице **women**. Это разрешение вида:

[women.change_women](#)

Для этого в каждый шаблон фреймворк Django автоматически передает коллекцию **perms**, содержащую разрешения текущего пользователя (если он авторизован). Например, если в шаблоне **women/post.html** в блоке **breadcrumbs** записать:

```
{{ perms }}
```

то должны увидеть объект **PermWrapper**. Но мы вместо него запишем следующую конструкцию:

```
{% if perms.women.change_women %}
<p><a href="{% url 'edit_page' post.slug
%}">Редактировать</a></p>
{% endif %}
```

Здесь проверяется наличие разрешения **women.change_women** и, если оно есть у пользователя, то отображается ссылка «Редактировать». Если это разрешение отсутствует, то ссылка появляться не будет.

Группы (Groups)

На практике нередко возникают ситуации, когда определенным пользователям нужно назначить один и тот же набор разрешений. Чтобы каждый раз не запоминать этот набор, его можно объединить в группу и, затем, пользователям назначать нужную группу (или несколько групп).

Группы можно создавать непосредственно в админ-панели:

<http://127.0.0.1:8000/admin/>

Например, создадим группу с именем **moderator** и поместим в нее все четыре разрешения для модели **women**. Затем одному из пользователей присвоим эту группу и удалим все дополнительные разрешения. Теперь при просмотре отдельных постов ему будет показываться ссылка «Редактировать», так как он обладает всеми четырьмя разрешениями, прописанными в группе. Если пользователь будет состоять в нескольких группах, то все их разрешения объединяются.

Команды работы с разрешениями и группами

Иногда при разработке проектов нам приходится работать с группами и разрешениями на уровне следующих команд:

```
myuser.groups.set([group_list])
myuser.groups.add(group, group, ...)
myuser.groups.remove(group, group, ...)
myuser.groups.clear()
myuser.user_permissions.set([permission_list])
myuser.user_permissions.add(permission, permission, ...)
myuser.user_permissions.remove(permission, permission, ...)
myuser.user_permissions.clear()
myuser.has_perm(permission)
```

Этот список взят из документации по адресу:

<https://docs.djangoproject.com/en/4.2/topics/auth/default/#permissions-and-authorization>

Здесь все достаточно очевидно. Откроем консоль фреймворка Django:

python manage.py shell

Импортируем модель **User** и прочитаем запись с **id=2**:

```
from users.models import User
user = User.objects.get(pk=2)
```

У этого пользователя есть внешние ключи **groups** и **user_permissions**:

```
user.groups
user.user_permissions
```

Соответственно, мы можем через них работать с группами и разрешениями. Например:

```
user.has_perm('women.add_women') # True
user.has_perm(women.delete_category') # False
```

Для добавления новых разрешений можно указывать или их **id**, например:

```
user.user_permissions.add(1)
user.user_permissions.all()
```

или объекты:

```
from django.contrib.auth.models import Permission
p = Permission.objects.get(codename='add_category')
user.user_permissions.add(p)
```

То же самое с группами:

```
from django.contrib.auth.models import Group
g = Group.objects.get(name='moderator')
user.groups.add(g)
```

или

```
user.groups.add(1)
```

Соответственно, мы можем удалять группы:

```
user.groups.remove(g)
```

или так:

```
user.groups.remove(1)
```


Очищать список групп:

```
user.groups.clear()
```

И то же самое с разрешениями:

```
user.user_permissions.remove(p)
user.user_permissions.remove(1)
user.user_permissions.clear()
```

Создание пользовательских разрешений

Иногда на практике возникает необходимость в создании собственных, дополнительных разрешений к списку уже существующих. В самом простом варианте это можно сделать через консоль Django следующим образом:

```
from django.contrib.auth.models import Permission
from django.contrib.contenttypes.models import
ContentType
content_type = ContentType.objects.get_for_model(User)
permission =
Permission.objects.create(codename="social_auth",
name="Social Auth", content_type=content_type)
```

Мы для модели **User** создали новое разрешение с кодовым именем **social_auth**, например, чтобы отмечать пользователей, авторизованных через социальные сети.

Если теперь посмотреть на список разрешений в окне админ-панели редактирования пользователей, то увидим новый пункт с именем **Social Auth** для модели **user** приложения **users**.

Задание

1. Создать приложение **users** для регистрации и авторизации пользователей.
2. При попытке доступа к определенной странице неавторизованного пользователя обеспечить его перенаправление на эту страницу после успешной авторизации.
3. Добавить возможность авторизации по E-mail адресу и паролю.
4. Создать страницу с профилем пользователя. Обеспечить возможность смены пароля в профиле.
5. Реализовать процедуру восстановления пароля с отправкой письма со ссылкой для восстановления пароля на реальный почтовый ящик. При переходе по ссылке из письма должна появляться форма изменения пароля, и после ее заполнения пароль должен быть изменен.

6. Создать группы разрешений и назначить некоторым пользователям разрешения, некоторым – группы. При необходимости добавить пользователей.
7. Создать пользовательское разрешение. Продемонстрировать его наличие в списке разрешений в окне админ-панели редактирования пользователей.